数值分析第一周上机作业

学号：241098117 ，姓名：张城宗

§1 题目一：机器精度、下溢值与上溢值

1.1 问题

分别针对 **双精度（double, float64）**和**单精度（single, float32）**浮点数，利用算法找出所使用计算机的机器精度 $\varepsilon_{\text{mach}}$、下溢值与上溢值，并编程验证算法的正确性，比较两种精度的差异.

1.2 算法思路

IEEE 754 标准规定了两种常用浮点格式，结构如下：

格式	总位数	符号位	指数位	尾数位	指数偏置
双精度 (double)	64	1	11	52	1023
单精度 (single)	32	1	8	23	127

隐含位（Hidden Bit）. 规格化浮点数的形式为 $1.f \times 2^e$，整数部分的 1 对所有规格化数恒成立，因此 IEEE 754 **不实际存储该位**，约定其隐式存在——称为**隐含位**. 由此，虽然双精度只存储 52 位尾数，实际有效位数却是 $52+1=53$ 位，单精度为 $23+1=24$ 位. 三个极限的理论值由有效位数 p（含隐含位）和指数范围 $[e_{\min}, e_{\max}]$ 给出：

$$\varepsilon_{\text{mach}} = 2^{-p}, \quad x_{\min} = 2^{e_{\min}}, \quad x_{\max} = (2-2^{-p}) \cdot 2^{e_{\max}}.$$

浮点数的本质是二进制. 计算机内部存储的是 64 位（或 32 位）二进制串，并非十进制小数. MATLAB 显示的 `2.2204e-16` 只是将二进制值转换为十进制字符串以便人类阅读. 可用 `num2hex(1.0)` 查看其十六进制存储值（`3FF0000000000000`），还原后即 $1.000\cdots0_2 \times 2^0$. " 单/双精度" 描述的是二进制表示的位宽，与是否用十进制显示无关.

代入具体参数，得到表 1 中的理论值. 三个算法的结构对双精度和单精度**完全相同**，仅数据类型不同；在 MATLAB 中，对单精度只需将初始值用 `single()` 转换，算术运算便自动在单精度下进行.

算法一：机器精度

思路：$\varepsilon_{\text{mach}}$ 的定义是满足 $1.0+\varepsilon > 1.0$ 的最小正数. 从 $\varepsilon = 1$ 出发，不断折半，直到 $1+\varepsilon/2$ 在计算机中与 1 相等，此时的 ε 即为所求.

Algorithm 1 求机器精度

```
ε ← 1.0
while fl(1.0 + ε/2) > 1.0 do
    ε ← ε/2
end while
return ε
```

注：IEEE 754 默认采用**最近舍入**（round-to-nearest-even）：若真实值恰好落在两个相邻浮点数的

正中间，则舍入到尾数末位为偶数的那个．由此，任意实数 x 的浮点化误差满足

$$\frac{|\mathrm{fl}(x)-x|}{|x|} \le u, \quad u = \frac{\varepsilon_{\mathrm{mach}}}{2} = 2^{-53}\text{（双精度）},$$

u 称为**单位舍入误差**（unit roundoff）．我们的算法得到的 $\varepsilon_{\mathrm{mach}} = 2^{-52}$ 是 1 与右邻浮点数的间距；正因为最近舍入将距离不超过 $\varepsilon_{\mathrm{mach}}/2$ 的数归并到 1，实际误差上界比间距小一倍，即 $u = \varepsilon_{\mathrm{mach}}/2$．

算法二：下溢值（正规化边界检测）

思路：下溢值即最小正规格化浮点数 `realmin`，是正规化得以维持的边界．正规化数 x 的相对精度恰好为 $\varepsilon_{\mathrm{mach}}$，即 $x + x\varepsilon_{\mathrm{mach}} > x$；一旦 x 变为非规格化数，$x\varepsilon_{\mathrm{mach}}$ 会因小于最小可表示正数而舍入为零，导致 $x + x\varepsilon_{\mathrm{mach}} = x$，相对精度" 消失"．利用此性质检测正规化边界：

Algorithm 2 求下溢值（正规化边界检测）

```
x ← 1.0
while fl(x/2 + (x/2)·ε_mach) > x/2 do        ▷ x/2 仍是正规化数
    x ← x/2
end while
return x                                      ▷ x = realmin = 2^{e_min}
```

注：下溢值是正规化得以维持的临界点，而非计算机能表示的绝对最小正数；算法的终止条件正是正规化相对精度消失的瞬间．

算法三：上溢值（两阶段二分逼近）

思路：$x_{\max}$ 的二进制表示为 $1.\underbrace{11\cdots1}_{52}\times 2^{1023}$，即 $2^{1023}+2^{1022}+\cdots+2^{971}$．算法分两个阶段：

第一阶段：从 $x=1$ 出发，不断翻倍，找到最大的 2 的整数次幂 2^{1023}（再翻倍则溢出）．

第二阶段：令步长 $s = x/2$，依次尝试将 s 加入 x——若不溢出则接受，反之则放弃；每轮步长减半，直到 $s < 1$ 为止．这本质上是逐位填充尾数的 52 个二进制位 1．

Algorithm 3 求上溢值（两阶段）

```
x ← 1.0
while fl(2x) ≠ +∞ do
    x ← 2x                                    ▷ 第一阶段：找到 2^{1023}
end while
s ← x/2
while s ≥ 1 do
    if fl(x + s) ≠ +∞ then
        x ← x + s                             ▷ 第二阶段：接受当前位为 1
    end if
    s ← s/2
end while
return x
```

1.3 结果分析

运行程序，将双精度与单精度的算法结果分别与理论值和 MATLAB 内置常量对比，如表 1 所示．

表 1: 双精度与单精度浮点数极限值对比

量	精度	算法计算值	MATLAB 内置值	理论值
机器精度	双精度	2.2204×10^{-16}	$\varepsilon = 2.2204 \times 10^{-16}$	2^{-52}
	单精度	1.1921×10^{-7}	$\varepsilon = 1.1921 \times 10^{-7}$	2^{-23}
下溢值（最小正规化数）	双精度	2.2251×10^{-308}	`realmin`$= 2.2251 \times 10^{-308}$	2^{-1022}
	单精度	1.1755×10^{-38}	`realmin`$= 1.1755 \times 10^{-38}$	2^{-126}
上溢值	双精度	1.7977×10^{308}	`realmax`$= 1.7977 \times 10^{308}$	$(2 - 2^{-52}) \cdot 2^{1023}$
	单精度	3.4028×10^{38}	`realmax`$= 3.4028 \times 10^{38}$	$(2 - 2^{-23}) \cdot 2^{127}$

机器精度:两种精度下算法结果均与内置值完全吻合,相对误差为 0.单精度的机器精度约为 1.19×10^{-7}，比双精度（2.22×10^{-16}）大约 9 个数量级，这直接反映了两者尾数位数的差异（23 位 vs 52 位）.
下溢值：算法结果与 `realmin` 完全一致，相对误差为 0．算法在 $x/2$ 失去相对精度 $\varepsilon_{\text{mach}}$ 时停止，精确捕捉到正规化的边界（双精度 2^{-1022}，单精度 2^{-126}）.
上溢值：两阶段算法对双精度和单精度均给出与 `realmax` 完全一致的结果，相对误差为 0．双精度的动态范围（$\sim 10^{308}$）远大于单精度（$\sim 10^{38}$），原因在于指数位数不同（11 位 vs 8 位）.

1.4 结论

三个算法均以" 折半/翻倍 + 临界条件" 为核心,对双精度和单精度**通用**,仅需切换数据类型（MATLAB 中用 `single()` 转换），结果与 IEEE 754 理论值完全吻合.

对比两种精度，关键差异如下：

- **精度**：双精度机器精度 $\approx 2.2 \times 10^{-16}$，单精度 $\approx 1.2 \times 10^{-7}$，相差约 9 个数量级；
- **动态范围**：双精度可表示至 $\sim 10^{\pm 308}$，单精度至 $\sim 10^{\pm 38}$；
- **存储开销**：单精度占用内存仅为双精度的一半（32 位 vs 64 位）.

实际应用中，选择精度需权衡：对精度和动态范围要求高的科学计算一般用双精度；深度学习等对速度和内存敏感的场景则常用单精度甚至更低精度.

§2 题目二：避免相减相消计算 $y = x - \sin x$

2.1 问题

设计算法计算 $y = x - \sin x$，使得有效位的丢失最多 1 位，并编程验证所设计算法的正确性.

2.2 算法思路

相减相消（Cancellation）现象. 当 x 接近 0 时，$\sin x \approx x$，两者几乎相等，直接相减会引发**相减相消**：两个相近的数之差虽然绝对误差不大，但结果的相对误差可极大，导致大量有效位丢失.

由讲义**精度丢失定理（定理 1.3.1）**：设 $x, y > 0$ 为规格化二进制浮点数且 $x > y$，若

$$2^{-q} \le 1 - \frac{y}{x} \le 2^{-p},$$

则做减法 $x - y$ 时至少丢失 p 个、至多丢失 q 个二进制精度位.

误差估计. 对 $y = x - \sin x$，以 x 为大数、$\sin x$ 为小数，利用 $\sin x$ 的 Taylor 展开得

$$1 - \frac{\sin x}{x} = \frac{x^2}{6} - \frac{x^4}{120} + \cdots \approx \frac{x^2}{6} \quad (x \to 0).$$

因此直接计算约丢失 $\log_2\left(\frac{6}{x^2}\right)$ 个二进制位.

阈值的确定. 要求直接公式至多丢失 1 位，需

$$1 - \frac{\sin x}{x} \;\ge\; 2^{-1} = 0.5 \iff \frac{\sin x}{x} \le \frac{1}{2}.$$

数值求解方程 $\sin x_0 / x_0 = 1/2$，得 $x_0 \approx 1.895$. 当 $|x| \ge x_0$ 时，直接公式至多丢失 1 位；当 $|x| < x_0$ 时，需使用改进算法.

改进算法：Taylor 级数. 将 $\sin x$ 展开并代入：

$$x - \sin x = \frac{x^3}{3!} - \frac{x^5}{5!} + \frac{x^7}{7!} - \cdots = \sum_{k=1}^{\infty} \frac{(-1)^{k+1}\, x^{2k+1}}{(2k+1)!}.$$

此级数各项均为有效加法，不存在相减相消，对小 x 收敛极快（首项 $x^3/6$ 即为主项）. 相邻两项的递推关系为

$$t_{k+1} = t_k \cdot \frac{-x^2}{(2k+2)(2k+3)},$$

逐项累加至 $|t_k| \le |y| \cdot \varepsilon_{\text{mach}}$ 时停止，即可达到机器精度.

综合两种情形，改进算法如下：

Algorithm 4 无相减相消地计算 $y = x - \sin x$

Require: x，切换阈值 $x_0 \approx 1.895$

Ensure: $y = x - \sin x$（至多丢失 1 个二进制位）

```
if |x| < x_0 then
    t ← x^3/6,   y ← t,   k ← 1
    while |t| > |y| · ε_mach do
        t ← t · (−x^2)/[(2k+2)(2k+3)],   y ← y + t,   k ← k+1
    end while
else
    y ← x − sin x                                   ▷ |x| ≥ x_0，至多丢失 1 位
end if
return y
```

2.3 结果分析

直接公式的精度损失. 以 Taylor 级数结果为参考真值，表 2 展示了直接公式 $y = x - \sin x$ 在不同 x 下的相对误差.

表 2: 直接公式与 Taylor 级数的对比（相对误差 $\approx 12\varepsilon_{\text{mach}}/x^2$）

x	直接公式	Taylor 级数（精确）	相对误差
10^{-2}	1.66667×10^{-7}	1.66667×10^{-7}	$\sim 10^{-12}$
10^{-4}	1.66667×10^{-13}	1.66667×10^{-13}	$\sim 10^{-8}$
10^{-6}	1.66533×10^{-19}	1.66667×10^{-19}	$\sim 10^{-3}$
10^{-8}	0	1.66667×10^{-25}	1（完全失真）

相对误差的理论近似为 $12\varepsilon_{\text{mach}}/x^2$，在双对数坐标下呈斜率 -2 的直线，阈值线 $x = x_0 \approx 1.895$ 恰与误差曲线交于 $y \approx 3\varepsilon_{\text{mach}}$，即「至多丢失 1 个二进制位」的临界点；阈值右侧误差已降至机器精度量级，直接公式完全可用.

2.4 结论

直接计算 $x - \sin x$ 对小 $|x|$ 会产生严重的相减相消，丢失约 $\log_2(6/x^2)$ 个有效二进制位. 改进算法以 Taylor 级数替代直接公式，彻底消除相消现象，在全域精度均达到机器精度量级. 切换阈值 $x_0 \approx 1.895$ 由精度丢失定理严格导出，保证了改进算法至多丢失 1 个二进制位.

§3 题目三：积分递推公式的数值不稳定性

3.1 问题

计算积分序列

$$y_n = \int_0^1 x^n e^x \,\mathrm{d}x, \quad n \geq 0.$$

由分部积分可得递推公式

$$y_{n+1} = \mathrm{e} - (n+1)\, y_n.$$

编程验证按上述公式设计的正向递推算法不稳定，并给出稳定的替代算法.

3.2 算法思路

初始值与真值界.

$$y_0 = \int_0^1 e^x \,\mathrm{d}x = \mathrm{e} - 1 \approx 1.71828.$$

由于 $x \in [0,1]$ 时 $1 \leq e^x \leq \mathrm{e}$，有

$$\frac{1}{n+1} \leq y_n \leq \frac{\mathrm{e}}{n+1},$$

即 $y_n > 0$ 且单调趋近于 0.

正向递推的误差分析（不稳定）. 设 $\varepsilon_n = \tilde{y}_n - y_n$ 为第 n 步的累积误差. 由递推公式：

$$\varepsilon_{n+1} = -(n+1)\,\varepsilon_n \implies |\varepsilon_n| = n!\,|\varepsilon_0|.$$

即使初始误差 $|\varepsilon_0|$ 仅为机器精度量级（$\sim 2 \times 10^{-16}$），经 n 步后误差放大为 $n!\,|\varepsilon_0|$. 当 $n = 20$ 时，$20! \approx 2.4 \times 10^{18}$，误差已完全淹没真值——算法**不稳定**.

反向递推（稳定）. 由递推公式解出 y_n：

$$y_n = \frac{\mathrm{e} - y_{n+1}}{n+1}.$$

误差传播：$\varepsilon_n = -\varepsilon_{n+1}/(n+1)$，即每步误差**缩小** $n+1$ 倍. 取足够大的 N（如 $N = 40$），令 $\tilde{y}_N = 0$（起点误差 $\leq \mathrm{e}/(N+1) \approx 0.066$），反向递推到 $n = 0$ 时，误差缩小为 $|\varepsilon_0| \approx |\varepsilon_N|/N! \approx 0$——算法**稳定**.

两种算法的误差传播对比如下：

表 3: 正向与反向递推的误差传播对比

	正向递推	反向递推
递推方向	$n = 0 \to N$	$n = N \to 0$
误差传播	$\lvert\varepsilon_{n+1}\rvert = (n+1)\lvert\varepsilon_n\rvert$	$\lvert\varepsilon_n\rvert = \lvert\varepsilon_{n+1}\rvert/(n+1)$
增长规律	$\lvert\varepsilon_n\rvert = n!\,\lvert\varepsilon_0\rvert$（指数增长）	$\lvert\varepsilon_n\rvert \to 0$（指数衰减）
稳定性	**不稳定**	**稳定**

3.3 结果分析

表 4 展示了两种递推算法与 MATLAB `integral()` 参考真值的对比：

表 4: 积分 y_n 的正向/反向递推与参考真值对比

n	参考真值	正向递推计算值	反向递推计算值
0	1.718281828	1.718281828	1.718281828
5	0.024313660	0.024313660	0.024313660
10	0.016105880	$0.016108\cdots$（开始偏离）	0.016105880
15	0.013103970	$-12.17\cdots$（已完全错误）	0.013103970
20	0.011109440	2.7×10^6（溢出量级）	0.011109440

正向递推在 $n\approx12$ 时首次出现负值（真值始终为正），之后完全失真；反向递推在整个区间内与参考真值完全吻合，验证了稳定性分析的结论.

3.4 结论

正向递推公式 $y_{n+1}=\mathrm{e}-(n+1)y_n$ 数值不稳定，误差按 $n!$ 增长，实用中应**禁止使用**. 改用反向递推 $y_n=(\mathrm{e}-y_{n+1})/(n+1)$，从大 n 处的近似起点出发，误差逐步缩小，可得到全程精确的结果. 这一例子表明：递推公式的数学等价性不等于数值等价性，方向的选择决定了算法的稳定性.

§4 题目四：实数序列递推算法的稳定性

4.1 问题

考察由以下递推关系定义的实数序列 $\{x_n\}$：

$$x_{n+1} = \frac{13}{3}x_n - \frac{4}{3}x_{n-1}, \quad n = 1, 2, 3, \ldots$$

分别在两组初值下分析算法的数值稳定性：

- **情形一**：$x_0 = 1,\ x_1 = 1/3$
- **情形二**：$x_0 = 1,\ x_1 = 4$

4.2 算法思路

特征方程与通解．令 $x_n = r^n$ 代入递推关系，除以 r^{n-1}（$r \neq 0$），得特征方程

$$3r^2 - 13r + 4 = 0 \implies r = \frac{13 \pm 11}{6},$$

故两个特征根为 $r_1 = 4,\ r_2 = 1/3$．通解为

$$x_n = A \cdot 4^n + B \cdot \left(\frac{1}{3}\right)^n.$$

情形一的精确解与稳定性分析．代入初值 $x_0 = 1,\ x_1 = 1/3$，解得 $A = 0,\ B = 1$，故精确解

$$x_n = \left(\frac{1}{3}\right)^n \xrightarrow{n\to\infty} 0.$$

然而，浮点计算中 $x_1 = 1/3$ 存在舍入误差 $\delta \sim \varepsilon_{\text{mach}}$，等价于引入一个微小的 $r_1 = 4$ 分量扰动，使实际计算值为

$$\tilde{x}_n = \left(\frac{1}{3}\right)^n + \delta \cdot 4^n.$$

由于 4^n 指数增长，而真值 $(1/3)^n$ 指数衰减，相对误差

$$\frac{|\tilde{x}_n - x_n|}{|x_n|} = \delta \cdot 12^n \xrightarrow{n\to\infty} \infty.$$

算法**不稳定**（$r_1 = 4$ 为增长的" 寄生解"，被微小扰动激发）．

情形二的精确解与稳定性分析．代入初值 $x_0 = 1,\ x_1 = 4$，解得 $A = 1,\ B = 0$，故精确解

$$x_n = 4^n.$$

浮点扰动激发 $r_2 = 1/3$ 分量，使计算值为 $\tilde{x}_n = 4^n + \delta \cdot (1/3)^n$，相对误差

$$\frac{|\tilde{x}_n - x_n|}{|x_n|} = \delta \cdot \left(\frac{1}{12}\right)^n \xrightarrow{n\to\infty} 0.$$

算法**稳定**（扰动项 $(1/3)^n$ 相对于主项 4^n 迅速衰减）．

两种情形的对比如下：

表 5: 两种初值下的稳定性分析

情形	初值	精确解	主特征根	相对误差增长规律
一	$x_0 = 1, x_1 = 1/3$	$(1/3)^n \to 0$	$r_2 = 1/3$（衰减）	$\delta \cdot 12^n \to \infty$（**不稳定**）
二	$x_0 = 1, x_1 = 4$	$4^n \to \infty$	$r_1 = 4$（增长）	$\delta \cdot (1/12)^n \to 0$（**稳定**）

4.3 结果分析

表 6 展示了两种情形的数值计算结果与精确解的比较：

表 6: 两种初值下的数值计算值、精确值与相对误差

n	情形一：$x_n = (1/3)^n$（应趋于 0）		情形二：$x_n = 4^n$（应趋于 ∞）	
	递推计算值	相对误差	递推计算值	相对误差
5	4.115×10^{-3}	$\sim 10^{-11}$	1024	$\sim 10^{-13}$
10	1.694×10^{-5}	$\sim 10^{-5}$	1048576	$\sim 10^{-15}$
20	-4.8×10^{-3}（负值！）	$\sim 10^{5}$	1.10×10^{12}	$\sim 10^{-15}$
30	1.7×10^{4}（完全发散）	$\gg 1$	1.15×10^{18}	$\sim 10^{-14}$

情形一在 $n \approx 18$ 时首次出现负值（真值恒为正），此后完全发散；情形二在整个计算过程中相对误差维持在机器精度量级，算法完全稳定.

4.4 结论

同一递推公式在不同初值下可以表现出截然不同的数值稳定性，其根本原因在于特征根的大小关系：

- 若精确解对应**较小的特征根**（如 $|r_2| = 1/3$），任何浮点扰动都会激发较大特征根（$|r_1| = 4$）对应的" 寄生解" 4^n，导致误差以 12^n 速率爆炸增长——**算法不稳定**；
- 若精确解对应**较大的特征根**（如 $|r_1| = 4$），扰动激发的小特征根分量 $(1/3)^n$ 相对于主项迅速衰减——**算法稳定**.

一般规律：对于线性递推方程，若算法要求追踪的解分量对应**非主导特征根**（绝对值较小的根），则数值误差将被主导特征根的增长所放大，产生不稳定性；只有当追踪的解分量本身是**主导分量**时，算法才具有数值稳定性.

§5 附录：程序代码

5.1 题目一

Listing 1: problem1_machine_precision.m（核心部分）

```
clear; clc;

%% --- 双精度 ---
% 机器精度
eps_calc = 1.0;
while (1.0 + eps_calc / 2) > 1.0
    eps_calc = eps_calc / 2;
end

% 下溢值：正规化边界检测（x/2 失去相对精度 eps 时停止）
underflow_calc = 1.0;
while underflow_calc/2 + (underflow_calc/2)*eps > underflow_calc/2
    underflow_calc = underflow_calc / 2;
end

% 上溢值（两阶段）
overflow_calc = 1.0;
while ~isinf(overflow_calc * 2),  overflow_calc = overflow_calc * 2;
    end
step = overflow_calc / 2;
while step >= 1
    if ~isinf(overflow_calc + step),  overflow_calc = overflow_calc +
        step;  end
    step = step / 2;
end

fprintf('[双精度] eps=%.4e, underflow=%.4e, overflow=%.4e\n', ...
    eps_calc, underflow_calc, overflow_calc);

%% --- 单精度：仅需把初始值换为 single() ---
eps_s = single(1.0);
while single(1.0) + eps_s / single(2.0) > single(1.0)
    eps_s = eps_s / single(2.0);
end

underflow_s = single(1.0);
while underflow_s/single(2) + (underflow_s/single(2))*eps_s >
    underflow_s/single(2)
    underflow_s = underflow_s / single(2);
end

overflow_s = single(1.0);
while ~isinf(overflow_s * single(2.0)),  overflow_s = overflow_s *
    single(2.0);  end
```

```
step_s = overflow_s / single(2.0);
while step_s >= single(1.0)
    if ~isinf(overflow_s + step_s),  overflow_s = overflow_s + step_s;
        end
    step_s = step_s / single(2.0);
end

fprintf('[单精度] eps=%.4e, underflow=%.4e, overflow=%.4e\n', ...
    eps_s, underflow_s, overflow_s);
```

5.2 题目二核心代码

Listing 2: problem2_avoid_cancellation.m（核心部分）

```
THRESHOLD = 1.895;  % 阈值: sin(x)/x = 0.5 的数值解

% 改进算法: 根据 |x| 与阈值自动切换公式
function y = improved_formula(x, threshold)
    if abs(x) < threshold
        y = taylor_series(x);   % 小 x: Taylor 级数, 无相减相消
    else
        y = x - sin(x);         % 大 x: 直接公式, 至多丢失 1 位
    end
end

% Taylor 级数: x - sin(x) = x^3/6 - x^5/120 + ...
function y = taylor_series(x)
    if x == 0;  y = 0;  return;  end
    term = x^3 / 6;    % 首项 k=1
    y = term;
    k = 1;
    while abs(term) > abs(y) * eps
        term = term * (-x^2) / ((2*k+2) * (2*k+3));  % 递推到第 k+1 项
        y = y + term;
        k = k + 1;
        if k > 200; break; end
    end
end
```

5.3 题目三核心代码

Listing 3: problem3_integral_recursion.m（核心部分）

```
e_val = exp(1);
N_forward  = 25;   % 正向递推阶数
N_backward = 40;   % 反向递推起始阶数

%% 参考真值（MATLAB 自适应数值积分）
for n = 0:N_forward
```

```
    y_true(n+1) = integral(@(x) x.^n .* exp(x), 0, 1, 'RelTol', 1e-15);
end

%% 正向递推: y_{n+1} = e - (n+1)*y_n (不稳定)
y_forward(1) = e_val - 1;      % y_0 = e - 1
for n = 0 : N_forward - 1
    y_forward(n+2) = e_val - (n+1) * y_forward(n+1);
end

%% 反向递推: y_n = (e - y_{n+1}) / (n+1) (稳定)
y_backward(N_backward + 1) = 0;    % 起点近似: y_N ~= 0
for n = N_backward - 1 : -1 : 0
    y_backward(n+1) = (e_val - y_backward(n+2)) / (n+1);
end
```

5.4 题目四核心代码

Listing 4: problem4_sequence_stability.m（核心部分）

```
c1 = 13/3;    c2 = 4/3;    N = 60;

%% 情形一: x0=1, x1=1/3, 精确解 x_n = (1/3)^n
x_c1(1) = 1;    x_c1(2) = 1/3;
for n = 2:N
    x_c1(n+1) = c1 * x_c1(n) - c2 * x_c1(n-1);
end
x_exact_c1 = (1/3).^(0:N);    % 精确解

%% 情形二: x0=1, x1=4, 精确解 x_n = 4^n
x_c2(1) = 1;    x_c2(2) = 4;
for n = 2:N
    x_c2(n+1) = c1 * x_c2(n) - c2 * x_c2(n-1);
end
x_exact_c2 = 4.^(0:N);        % 精确解
```